

BSDS · Midterm Mastery

# Building Scalable Distributed Systems

Parts I · II · III

Srimansi Rameshkumar · CS 6650

# Today's Agenda



## Assignment 6 Results

Breaking point, ALB scaling & resilience testing - shared on Piazza



## Course Reflections

5 concepts that reshaped how I think about distributed systems



## Crash & Recovery

Cascading failure demo + circuit breaker fix

Part I

# Assignment 6

# Results

Breaking point → ALB scaling → Resilience

## A6 Part II: Single Task Breaking Point

Setup: Single ECS Fargate task (256 CPU / 512MB) · 300 rounds SHA-256 per request to simulate CPU-intensive work · Locust with constant(0) wait time

| Users | RPS | Median | CPU  | Memory | Observation                      |
|-------|-----|--------|------|--------|----------------------------------|
| 5     | ~20 | ~10ms  | ~50% | ~44%   | Healthy, plenty of headroom      |
| 20    | ~20 | ~12ms  | ~89% | ~44%   | CPU saturated, RPS plateaus      |
| 50    | ~20 | ~15ms  | ~89% | ~44%   | Same plateau, CPU is the ceiling |
| 100   | ~20 | ~18ms  | ~89% | ~44%   | More users, no more throughput   |

Key Finding: RPS plateaued at ~20 regardless of user count. CPU was the bottleneck, not code logic. Memory stayed stable at 44% throughout. The fix is more compute, not better code.

## A6 Part III: ALB + Auto Scaling

Setup: ALB + Auto Scaling · min 2 tasks, max 4 · scale out at 70% CPU · same SHA-256 load

### Load Test Results

| Users | RPS | Tasks | CPU  |
|-------|-----|-------|------|
| 5     | ~20 | 2     | ~45% |
| 20    | ~40 | 2     | ~80% |
| 50    | ~60 | 3     | ~70% |
| 100   | ~80 | 4     | ~68% |

### Resilience Test

Mid-load: manually stopped  
one running task

**17,607**

Total requests

**0**

Failures

**< 30s**

Recovery time

*Key Finding: RPS scales linearly with task count (~20 RPS/task). ALB reroutes traffic seamlessly when a task fails.*

Part II

# Course Reflections

5 concepts that reshaped my thinking

# 5 Concepts That Shaped My Thinking

01

## Modular Decomposition

Parnas (1972): hide design decisions inside modules. Changing one module shouldn't ripple across the system.

02

## Logical Clocks

Lamport's happened-before relation orders events in distributed systems without relying on physical time.

03

## Load Testing

Empirically find breaking points: CPU, memory, goroutines. You can't optimize what you haven't measured.

04

## Horizontal Scaling

ALB + Auto Scaling: RPS scales linearly with task count. CPU saturation means add tasks, not bigger machines.

05

## Resiliency Patterns

Fail fast + circuit breaker: isolate dependency failures so one slow service can't take down the whole system.

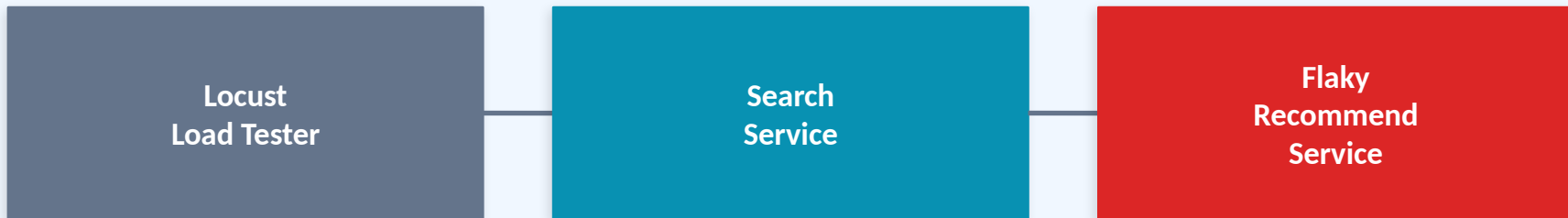
Part III

# Crash & Recovery

Cascading failure → Circuit breaker → Fail fast



# Experiment Setup



## Flaky Recommend Service vs Search Service Response

- Randomly sleeps 2-10 seconds, simulates slow ML model inference
- 30% random failure rate on top of the delay
- Without protection: goroutines block indefinitely, memory climbs, CPU stays near 0%
- With protection: 500ms timeout cuts it off; circuit breaker opens after 5 failures and stops all downstream calls for 30s
- Toggle between modes via `PROTECTION_ENABLED` env var, no redeployment needed

# Load Testing Results: Both Phases

## Phase 1: No Protection

| Users | RPS  | Median  | CPU  | Mem  |
|-------|------|---------|------|------|
| 5     | ~1.0 | 5,100ms | 0.5% | 5.4% |
| 20    | ~3.6 | 5,400ms | 2%   | 9.9% |
| 50    | ~8.6 | 5,200ms | 2%   | 5.0% |

**Memory > CPU** throughout. Goroutines blocked on I/O, not computing

*More users just means more blocked goroutines. The service is in a permanent slow crawl with no traditional breaking point.*

## Phase 2: With Protection

| Users | RPS | Median  | CPU | Mem |
|-------|-----|---------|-----|-----|
| 5     | ~27 | 130ms   | 5%  | 8%  |
| 20    | ~25 | 430ms   | 22% | 8%  |
| 50    | ~28 | 1,000ms | 46% | 10% |

**CPU > Memory** - service is doing real compute again

*At 50 users CPU hits 46%. Bottleneck shifts back to compute. The next fix is horizontal scaling, just like A6.*

# Key Takeaways

1

## Scaling & Resiliency solve different problems

A6: CPU saturation: add more tasks. A7: slow dependency: add circuit breaker. Both are needed; neither replaces the other.

2

## Memory > CPU is the signature of an I/O-bound failure

Goroutines blocked waiting on a slow downstream sit idle in memory. Adding more compute does not help. You need a code-level boundary.

3

## Fail fast, not fail slow

A 500ms error is better than a 9-second hang. Degrade gracefully: serve search results without recommendations rather than serving nothing at all.